

Un problème d'éclairage

Matin, Jordan

May 7, 2024

Contents

1	Introduction	3
2	Méthodes	3
2.1	Quelques notations	3
2.2	Déroulement	3
2.3	Choix des structures de données	4
2.3.1	Représentation de graphe	4
2.3.2	Lecture du fichier	5
2.3.3	Choix des parcours de graphes	5
3	Résultats	6
3.1	Peut-on allumer toutes les ampoules en même temps ?	6
3.1.1	Graphes connectés et composantes connexes et fortement connexes	6
3.1.2	Complexité	6
3.1.3	Algorithme de décision pour la question 1	8
3.1.4	Algorithme alternatif	10
3.2	Combien d'ampoules peut on allumer au maximum ?	10
3.2.1	Pas de réponse en temps polynomiale?	10
3.2.2	Algorithme	11
3.2.3	Heuristique	11

1 Introduction

Nous nous intéressons au problème suivant : Étant donné un tableau $n \times n$ avec un interrupteur pour chaque ligne et chaque colonne placé respectivement en vertical et en horizontal, p ampoules (et donc au maximum n^2 ampoules), peut-on allumer simultanément toutes les ampoules ? Sinon, combien peut-on en allumer au maximum ?

Pour répondre, nous avons mobilisé nos connaissances du cours d'algorithmique 2 ainsi que du cours d'informatique fondamentale INFO F302, et aussi à l'aide de ressources extérieures, comme Wikipedia.

Répondre à la première question n'est pas une tâche aisée que si l'on ne prend pas en compte la complexité : il suffit de tester chaque combinaison d'interrupteurs jusqu'à obtenir confirmation. Difficulté oubliée, nous y répondons avec une complexité linéaire en temps. En revanche, répondre à la deuxième question ne peut se faire en temps linéaire, ni même en temps polynomial, en général. Nous allons nous intéresser, dans le cadre de ce rapport, à différents aspects des deux problèmes et allons identifier les sous-problèmes sous-jacents ; parmi ceux-ci, nous allons notamment analyser la complexité inférieure et lui donner une borne, la complexité moyenne, ainsi que la complexité maximale et lui donner une borne supérieure.

Les réponses, évidemment, sont intimement liées à la disposition et aux nombres d'ampoules sur le tableau d'interrupteur.

2 Méthodes

2.1 Quelques notations

Nous prenons comme conventions que les interrupteurs situés sur le flanc gauche de l'image du projet, c'est-à-dire agissant sur les lignes, seront notés (Y,F) ou (Y,V) selon la position ouverte ou fermée, et nous noterons pour les interrupteurs contrôlant les ampoules verticalement par : (X,F) ou (X,V) .

"VF" : Signifie l'ampoule s'allume si et seulement si l'interrupteur Y est ouvert et l'interrupteur X est fermé. "VF,"FV" : Signifie l'ampoule s'allume si Y est fermé et X est ouvert et vice versa ou si Y est fermé et X est ouvert et vice versa. Lorsque nous disons "combinaison triviale" nous sous-entendons qu'il s'agit d'une ampoule ne s'allumant que pour une seule combinaison ou bien s'allumant pour deux combinaisons où seulement un seul des deux interrupteurs doit être dans une certaine position. Lorsque nous créons une arête entre deux noeuds, c'est équivalent à ce que l'ampoule correspondantes est allumée; ainsi les arêtes réciproques ne sont pas comptabilisées lors de la vérification de notre algorithme, car ceci fausserait le nombre d'ampoule allumées.

2.2 Déroulement

Nous décrivons brièvement notre fonctionnement et méthodologie pour la réalisation du projet

1) Identification des composantes connexes

(2) Appliquer l'algorithme sur les composantes connexes dans l'ordre croissant : c'est-à-dire que si l'on parvient à montrer une contradiction dans la plus petite composante connexe, nous montrons en particulier une contradiction dans l'ensemble des composantes connexes et donc que nous ne pouvons pas allumer toutes les ampoules en même temps.

(3) Si nous avons affaire à un graphe non séparé, nous pouvons alors partir d'une arête et parcourir le graphe jusqu'à obtenir une contradiction. Le cas échéant, nous remarquons qu'il n'est pas suffisant pour conclure : en effet, les interrupteurs peuvent se trouver dans l'un ou l'autre état, et si par hypothèse, l'ensemble des ampoules sont allumées simultanément, alors nécessairement, l'interrupteur doit se trouver dans une position permettant à l'ampoule de s'allumer et donc de facto il se pouvait qu'il s'agissait de l'autre combinaison,

et dans ce cas il suffit de reparcourir à partir du noeud opposé. Nous avons donc un algorithme en temps linéaire pour résoudre la première question.

En ce qui concerne la question deux, nous développons une approche "heuristique", c'est-à-dire que nous développons certains cas théoriques pour lesquels notre approche n'est pas exponentielle, et nous montrons ensuite rigoureusement qu'il n'est en général, pas possible de produire un algorithme en temps polynomial pour résoudre la question. Ceci motive le choix opté pour utiliser un backtracking, qui dans notre cas, est donc l'algorithme dit "brute force" afin de se persuader d'une réponse maximale. Nous avons également pensé à utiliser la question 1 sur $n-1$ ampoules, et vérifier s'il existe une configuration des interrupteurs telle que toutes les ampoules sauf une seraient allumées, et décrémenter n sinon. Nous y reviendrons plus tard.

Nous avons fait le choix d'inclure des screenshots de morceaux de codes parfois en Python et parfois en Java ou les deux en même temps, selon la lisibilité du code. Il est à noter que seule la question 1 du projet a pu être correctement programmée en Java (bien que nous avons programmé la totalité du code en Python), dû au fait que nous avons rencontré plusieurs difficultés techniques en cours de route : le code du cours de référence ne dispose pas toujours d'un package complet, par exemple permettant au lancement de la classe "Di graph", ce qui a créé beaucoup de contretemps conséquents. De plus, nous avons rencontré une boucle infinie pour la question 2 (en Java) ce qui nous a poussés à écrire la totalité du code en Python. Nous avons opté pour ce choix de liberté car il s'agit d'un cours d'algorithmique et que nous ne sommes pas réellement testés sur nos compétences en programmation ; évidemment tout ceci ne justifiera pas le manquement aux consignes et nous nous en excusons à l'avance.

Ceci dit, nous définirons plusieurs notions essentielles, qui ont été capitales, quant à la décision de la résolution de la question 2 par un backtracking ; nous avons longtemps songé à reproduire un algorithme de temps linéaire voire polynomial. Nous allons dans cette optique reproduire nos cheminement et décortiquer pas à pas comment nous avons abordé le problème. Nous parlerons entre autres de : plus long chemin, k clique, de réductions, de plus court chemin dans un DAG (un graphe dirigé acyclique) et comment cette dernière notion a occupé notre esprit durant près de trois semaines. Nous allons appuyer nos résultats par plusieurs illustrations, où nous prendrons la liberté de les dessiner au stylo. Nous avons opté pour un graphe représenté sous forme de dictionnaire équivalent à une liste d'adjacence. Ce choix a été motivé à la fois par le fait que sont ainsi définies les implémentations de graphe dans le livre de référence (même si ce sont des listes d'adjacence) mais aussi pour les nombreux cas où nous aurons à faire à une densité d'ampoules faible voire négligeable comparée aux nombres d'interrupteurs. En effet, nous soulageons la machine en termes d'espaces, car nous évitons d'occuper un espace de l'ordre carré N^2 toutes situations confondues. Nous reviendrons avec plus de détails sur le choix de cette implémentation plus loin.

2.3 Choix des structures de données

2.3.1 Représentation de graphe

Nous avons choisi une représentation sous forme de liste d'adjacence/dictionnaire, car nous aurons affaire à maximum $2I$ noeuds ou I est le nombre d'interrupteur; en effet chaque interrupteur possède deux états. Par ailleurs les interrupteurs ne pouvant être que dans une position à la fois, nous avons trouvé judicieux de manipuler des dictionnaires avec pour clés les dits noeuds, et comme valeurs soit "F" soit "V" selon le parcours réalisé. Ensuite, nous jouissons également de la lisibilité en ce qui concerne les arêtes : puisque nous avons à faire à un graphe dirigé, il est dès lors aisé de vérifier dans quel sens va la flèche, et quels sont les arêtes et les voisins d'un noeud. Nous soulignons que nos listes d'adjacences sont en fait des dictionnaires avec pour clé un noeud et comme valeur une liste Python/Java contenant ses voisins, par exemple : $\{(Y2,V) : [(X1, F), (X5, F)]\}$.

2.3.2 Lecture du fichier

La lecture du fichier était un sérieux défi. Il n'est pas trivial de faire correspondre les "1" et "0" avec des arêtes : pour rappel, une arête entre deux nœuds représente une implication du type : Supposition (i) : Toutes les ampoules sont allumées simultanément, Supposition (ii) : l'interrupteur v est en position Y. Implication : l'interrupteur u est en position X. De ceci, nous nous retrouvons avec des cas dits triviaux : c'est-à-dire si une ampoule ne peut être allumée que pour une seule combinaison, alors nécessairement si l'ampoule est allumée, les deux interrupteurs sont nécessairement dans l'unique paire de positions qui allume l'ampoule. Nous soulignons néanmoins que nous ne traiterons pas ces cas comme des cas triviaux au sens de la non-inclusion de ces derniers dans notre graphe. Nous y reviendrons avec plus de détails dans la section résultats, ainsi que dans la section résultats de la partie 2. À nouveau, nous classons d'autres cas comme triviaux : par exemple : "FF, FV", "VV, VF", "VF, VV", puisque l'éclairage de l'ampoule ne dépend plus que d'un unique interrupteur. En effet, peu importe la position de l'un, il suffit que l'autre soit dans une certaine position. Nous restons donc avec le deuxième groupe des caractères : "VF, FV" et "VV, FF", ainsi que les 4 combinaisons possibles à 3 unités dans une ligne du fichier, dits "non triviaux" car l'implication prend entièrement son sens. Les deux premiers sont clairs, et les détails des 4 derniers sont laissés dans l'implémentation Python/Java. Nous détaillons seulement le fait qu'il n'y a pas d'autres possibilités hormis les cas profondément triviaux, c'est-à-dire : "0000" et "1111". En effet, nous disposons de 4 manières d'avoir des combinaisons de 3 unités par la formule $\binom{4}{3} = \frac{24}{6} = 4$.

2.3.3 Choix des parcours de graphes

Sans surprise, nous avons opté pour les deux parcours de graphe les plus classiques : le parcours en profondeur (DFS) et en largeur (BFS). Nous explorons avec le DFS les composantes dans un premier temps pour savoir s'il y a des composantes connexes. Par ailleurs, nous avons omis d'implémenter la vérification des lignes et des colonnes des ampoules afin de vérifier s'il n'existe pas d'ampoules isolées ou en cluster. Le BFS est au centre de ce projet, car nous nous en servons à la fois pour détecter les nœuds contradictoires et à la fois pour ne pas explorer les branches venant de nœuds contradictoires "à tour de rôle". Nous avons préféré ce dernier, surtout dans la question 1, car nous avons une connaissance directe des contradictions les plus proches, ce qui en moyenne nous fait gagner en termes de complexité. Voici à quoi ressemble le graphe associé au fichier :

Table 1: Exemple de fichier

1	2	0	1	1	0
1	3	1	0	0	0
2	1	0	0	1	1
2	2	0	1	1	0
2	4	1	0	0	1
3	1	0	1	1	0
3	3	1	0	0	1
3	4	1	0	1	0
4	1	1	0	0	1
4	2	1	1	0	0
4	3	0	1	1	0

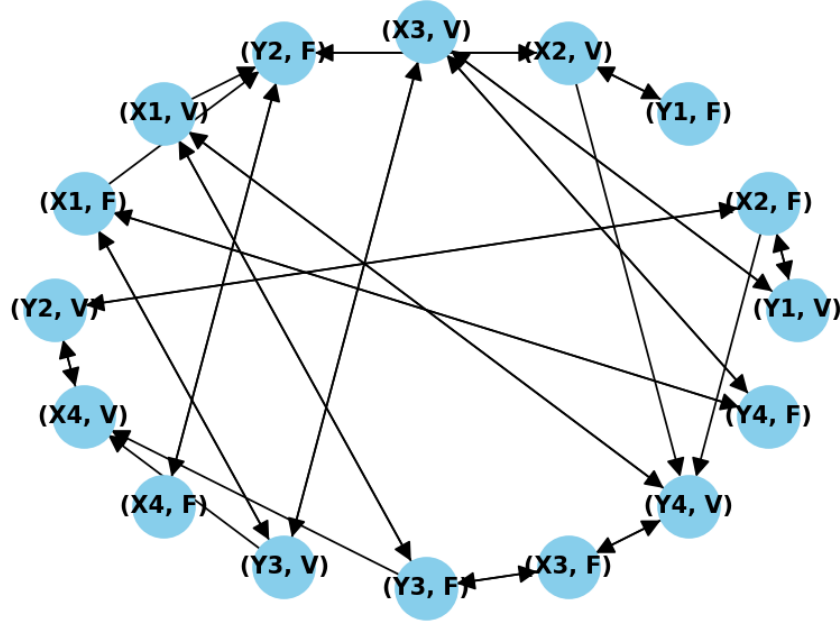


Figure 1: Graphes des implications

3 Résultats

3.1 Peut-on allumer toutes les ampoules en même temps ?

3.1.1 Graphes connectés et composantes connexes et fortement connexes

(i) Nous commençons par remarquer que certaines dispositions comme celle illustrée dans l'image de l'énoncé du projet sont moins coûteuses en temps : si nous analysons les nœuds ici, non plus comme des interrupteurs mais plutôt comme des ampoules, et que nous imaginons des arcs entre chaque ampoule si et seulement si deux ampoules sont alignées sur une même ligne ou une même colonne (ou les deux) le tout formant un graphe non dirigé. Clairement, dans l'exemple, il y a deux composantes connexes et donc les ampoules peuvent être allumées simultanément si et seulement si nous arrivons à allumer les deux composantes connexes.

Ainsi notre complexité est réduite, puisque si notre algorithme de décision renvoie False sur l'une des deux composantes connexes, il renverra False sur le graphe en entier (graphes des implications). Dans cette optique, il convient donc d'analyser la composante connexe la plus petite possible (en termes d'ampoules). De ceci, si nous avons n ampoules sur la diagonale du tableau, puisqu'il n'existe aucun arc les joignant, elles sont toutes indépendantes, et donc de facto elles peuvent toutes être allumées simultanément. De ceci si cette information est connue, notre algorithme serait donc en $O(1)$. Toutefois, afin de vérifier que les ampoules seraient toutes indépendantes, il faudrait d'abord réaliser de l'ordre de I^2 opérations : en effet il faut effectuer $I/2$ passes pour se rendre compte qu'il n'y a pas 2 ampoules sur la même ligne, et puis répéter $I/2 \times (I/2 - 1)$ passes pour s'assurer qu'il n'y a pas deux ampoules sur la même colonne. Nous discutons rigoureusement de la complexité en temps dans la section suivante.

3.1.2 Complexité

La première étape est la lecture du fichier.txt donné en entrée du programme et l'extraction des données. Nous supposons que le fichier est écrit sur F lignes, les colonnes elles, sont fixées et il y en a exactement six. Nous utilisons la fonction `ones_zeros_to_VF_in_dico` pour lire et créer un dictionnaire contenant comme

clé un tuple contenant les deux premières entrées de chaque ligne du fichier, et comme valeurs le reste des entrées pour chaque ligne, où nous avons converti les "zéros et les uns" en string : "FV", "VV" etc... et nous créons la variable `dico_ampoules_vrai_faux`. Voici à quoi ressemble cette dernière avec un exemple de fichier, cf. le fichier README :

$$dico_ampoules_vrai_faux = \{(1, 3) : ['VV', 'VF', 'FV'], (2, 1) : ['VV', 'VF', 'FF'], (2, 2) : ['VF', 'FV']\}$$

Nous sommes donc en $O(6F)$. Nous discutons maintenant la complexité de la création du graphe. Puisque nous utilisons une représentation équivalente à une liste d'adjacence, nous sommes en $O(S + E)$ où S représente les sommets (interrupteurs en position ON et OFF) et E les arêtes, en termes d'espace. Par ailleurs, avant la création du graphe il nous faut discuter les différents scénarios pour créer une arête. Nous créons le graphe en nous basant sur le dictionnaire `dico_ampoules_vrai_faux` :

Nous considérons 3 cas : soit l'ampoule ne possède qu'une seule combinaison, soit deux, soit trois. Comme énoncé dans la section précédente, si l'ampoule s'allume sous quatre combinaisons, alors il s'agit d'une combinaison profondément triviale. Puisque chaque élément du dictionnaire `dico_ampoules_vrai_faux` est une clé correspondant à une ampoule, et qu'il y a au maximum 4 combinaisons pouvant être construites "en termes d'implication", c'est-à-dire si on ajoute une combinaison à "FF,VV", nous n'aurons pas une arête en plus mais simplement deux arêtes différentes. Ainsi pour une ampoule fixée, ceci livre donc une complexité en $O(4N)$ puisque la création d'une arête avec notre structure de donnée est négligeable.

Nous sommes donc pour l'instant en $O(4N + 6F)$ et donc de l'ordre linéaire.

Nous revenons brièvement sur la discussion du point (i) : Nous pourrions entrevoir un algorithme qui détecte si les ampoules sont disposées de manière à former plusieurs composantes connexes, simplement en lisant les lignes de haut en bas qui repère les combinaisons ne pouvant aboutir à l'allumage simultané de deux ampoules sur la même ligne ou colonne. Sur le tableau de l'exemple ci-dessous cf tableau 1, on voit clairement que les ampoules (1,3) (3,3) ne peuvent pas être allumées simultanément. Il aurait suffi seulement de parcourir les lignes jusqu'à ce que le deuxième élément soit égal à 3. Si l'on suppose que l'on avait une combinaison faisant allumer les deux ampoules, nous n'aurions qu'à remarquer que les ampoules (3,3) (3,2) ne peuvent pas être allumées simultanément. En effet les combinaisons "OO" & "FV,FF" rendent cela impossible.

Entre autres l'algorithme part de la première ligne du fichier et explore les ampoules situées sur la colonne et la ligne correspondante ce qui livre donc $N - 1 + N - 1 = 2N - 2$ passes pour une ampoule. S'il n'y a pas de contradiction il nous faut donc accéder à la prochaine ampoule non visitée et reproduire le même raisonnement. Dans tous les cas nous devons visiter N^2 ampoules et l'algorithme nous apporte une décision. Remarquons que la complexité est polynomiale et qu'il devient difficile de juger si contradiction il y a, lorsqu'une ampoule peut s'allumer sous différentes combinaisons. Remarquons que cet algorithme se réduit à un algorithme pour trouver les ampoules connexes et donc que nous sommes en $O(N^2)$ au maximum. En revanche la complexité minimale elle est de $O(N)$: en effet si on visite les éléments en première position dans le fichier, on peut se rendre compte que des ampoules sont situées sur la même ligne et s'il n'y en a qu'une seule nous sommes en $O(1)$ (les premiers éléments sont triés) mais auquel cas doit s'ajouter le parcours des n lignes du fichier en ce qui concerne les éléments en deuxième position i.e. les éléments situés sur la même colonne (non triés).

On en conclut qu'il est moins coûteux de vérifier si les ampoules situées sur la même ligne peuvent s'allumer en même temps. Ainsi si nous avons N ampoules alignées, nous pourrions appliquer notre algorithme sans même devoir créer de graphe. Toutefois, ceci ne peut s'appliquer que dans certains cas très précis et ne constitue donc pas un algorithme efficace en général, car discuter s'il y a des combinaisons contradictoires, cela devient très vite difficile.

1	3	0	0	0	1
2	1	1	0	1	0
2	4	0	1	1	0
3	2	0	0	1	1
3	3	1	0	0	0
4	1	0	1	0	0
4	4	0	1	1	0

Table 2:

3.1.3 Algorithme de décision pour la question 1

Nous discutons ici le fonctionnement de notre algorithme qui répond à la question de manière beaucoup plus générale. Le BFS constitue le squelette de notre algorithme : il parcourt le graphe jusqu'à ce qu'il y ait un noeud atteint qui contredit le principe de graphe d'implication. Supposons que nous démarrons à partir du noeud $u := (Y1, V)$ et que $a = (X2, F)$ et que nous ayons les implications $u \rightarrow a \rightarrow \dots \rightarrow v := (Y1, F)$.

Ceci est équivalent à dire : toutes les ampoules sont supposées allumées, et nous supposons que le noeud Y1 est en position ON, ce qui implique que le noeud X2 est en position OFF et ainsi de suite jusqu'à obtenir que le noeud Y1 est en position OFF. Nous avons donc à la fois que Y1 est en position ON et en position OFF, ce qui constitue une contradiction. Il est important de noter que trouver une telle contradiction ne constitue pas une preuve ! Avant d'aller plus en profondeur sur le fonctionnement complet de notre algorithme, nous allons prouver un résultat. Notre algorithme en sera un corollaire immédiat.

Lemme 1: Si N ampoules s'allument simultanément, alors il n'y a qu'une et une seule combinaison activée pour chaque ampoule.

- Preuve : Par définition, un interrupteur est soit en position On ou OFF, donc si une ampoule est allumée, ses deux interrupteurs associés sont tous les deux dans une et une seule position ce qui livre donc une seule combinaison.

Lemme 2: Si N ampoules sont allumées, il y a N paires de combinaisons actives.

- Preuve : Suit directement du lemme 1.

Théorème : Soit $I > 1$, soient I interrupteurs. N ampoules sont allumées simultanément si et seulement si pour tout noeud $u := (C_i, P_j)$ pour i dans $\{1, \dots, I/2\}$, avec C_i à valeur dans $\{X_i, Y_i\}$ et P_j à valeur dans $\{V, F\}$, il n'y a pas à la fois une contradiction en partant de u et une contradiction en partant de $u' := (C_i, P'_j)$ avec P'_j la position inverse de P_j . Par exemple, si $P_j = V$, alors $P'_j = F$.

- Preuve :

Par lemme 2, nous savons que si les ampoules sont allumées simultanément alors tous les interrupteurs sont dans une position fixée. Si une contradiction d'un noeud a lieu, ceci implique que ce noeud en question (qui est un interrupteur dans une position ouverte ou fermée) ne permet pas l'allumage simultané de toutes les ampoules. D'autre part, puisque les ampoules sont toutes allumées simultanément, ceci implique que l'interrupteur doit se trouver dans une position et la seule position restante est la position opposée. Ceci livre un sens.

Dans l'autre sens, si nous avons la garantie de n'avoir qu'au plus une contradiction pour chaque paire (u, u') de noeud, alors nous pouvons toujours choisir une position de l'interrupteur ne menant pas à un chevauchement de positions. En d'autre terme, il existe une combinaison des interrupteurs telle que l'on peut allumer toutes les ampoules simultanément.

Nous passons maintenant à une analyse profonde de notre algorithme. Commençons par remarquer qu'il est inutile de continuer à parcourir le graphe une fois que nous avons traversé autant de noeuds qu'il y a

d'interrupteurs. cf Lemme 1. Le compteur `count` vérifie que nous avons plus une contradiction en partant d'un noeud. cf Théoreme. Ceci est illustré par le bout de code ci-dessous :

```
if (visitedDico.size() == nombre_interrupteurs) {
    break;
}
while (!queue.isEmpty() && count < 2) {
```

Ainsi, si contradiction il y a, dans le pire des cas nous traversons au plus I nœuds, auquel cas il nous faut refaire un parcours depuis le nœud opposé, ce qui livre à nouveau au maximum I nœuds à traverser.

En ce qui concerne la vérification des contradictions, nous utilisons un dictionnaire : "visited_dico" où nous plaçons les nœuds visités au fur et à mesure. Après chaque passage d'un nœud, nous créons donc une clé avec la lettre et le numéro du nœud correspondant et comme valeur "V" ou "F" selon qu'il est en position ON ou OFF.

Notons que chaque clé n'a donc qu'au plus une seule valeur associée, et que chaque clé peut être accédée en $O(1)$ car Java utilise des "Hashmap".

Une autre considération importante est le fait de savoir si le graphe des implications est séparé. En effet, la possibilité de rencontrer une contradiction en cours de route, non plus au sein du sous-graphe connecté, mais lors du passage de la boucle `for` à un autre nœud juste après avoir terminé de parcourir le sous-graphe en question. Pour ce faire nous vérifions que " $i == 0$ " afin de se persuader que nous sommes toujours au sein même du sous graphe connecté. Si contradiction il y a, au sein du graphe connecté, il faut donc réinitialiser les valeurs parcourues ainsi que les arêtes visitées, comme illustré par le bout de code ci-dessous :

```
if (visitedDico.containsKey(currentNode.substring(1, 3))) {
    if (visitedDico.get(currentNode.substring(1, 3)) != currentNode.charAt(currentNode.length() - 2) &&
        visitedDico.clear();
        queue.clear();
        visitedEdges.clear();
        count++;
    }
}
```

Autrement, si la queue est vide, nous incrémentons la variable " i ", et nous passons au nœud suivant :

```
else if (visitedDico.get(currentNode.substring(1, 3)) != currentNode.charAt(currentNode.length() - 2) &&
    continue;
}
```

Enfin, remarquons qu'une fois le nombre maximal d'arêtes pouvant être construites uniquement en considérant les implications est atteint, il peut subsister des nœuds qui peuvent être reliés par une arête : ceci est dû aux ampoules ayant exactement trois combinaisons. Pour pallier à cela, nous traversons l'ensemble des ampoules ayant trois combinaisons et nous vérifions si les ampoules restantes peuvent être allumées. Cette étape supplémentaire nécessite de parcourir les ampoules ayant trois combinaisons, et vérifier si la position fixée des interrupteurs correspondant à ces ampoules permet d'ajouter une nouvelle arête.

Nous sommes donc en $O(K)$ où K est le nombre d'ampoules possédant 3 combinaisons, pour cette opération supplémentaire. Évidemment, cette opération se produit à la fin de notre algorithme pour certains cas seulement, et est indépendante en termes de complexité de la première partie de l'algorithme.

Pour conclure, nous effectuons entre autre un parcours en largeur qui est linéaire en temps $O(I+E)$ où E est le nombre d'arête et I le nombre d'interrupteurs au cours duquel nous effectuons des opérations constantes en temps. L'algorithme est en $O(2I + E + K)$ au maximum, ce qui donne la borne supérieure. La borne inférieure, dans le cas où l'on peut allumer toutes les ampoules, est en $O(I + E')$ avec $E' \leq E$ le nombre d'arêtes reliant les noeuds non contradictoires. La borne inférieure dans le cas où l'on ne peut pas allumer toutes les

ampoules, est en $O(8 + 4)$: deux ampoules alignées induisent quatre noeuds et au moins deux implications; en effet si contradiction il y a en partant d'un noeud, il faut repartir du noeud opposé. Evidemment, l'algorithme peut être optimisé pour réaliser de l'ordre de $O(1)$ opérations. cf section 3.1.1.

3.1.4 Algorithme alternatif

Les discussions précédentes prenaient en compte le fait d'ajouter des implications dites triviales. Nous aurions pu aussi ne pas les considérer. En effet, la création de ces arêtes est redondante et n'est pas nécessaire quant à la décision d'un algorithme en temps linéaire. Sans perte de généralité, si nous nous étions seulement donné des ampoules ayant au maximum 2 combinaisons, nous pouvons construire un algorithme alternatif de la sorte :

Nous construisons une arête si et seulement si s'il s'agit une implication non triviale. Ainsi ne sont donc pas créées les arêtes correspondant aux ampoules ayant pour combinaisons : "VV, VF", "VV, VF", "VF, FF", "FV, FF" ainsi que les ampoules ayant une seule combinaison. Il en ressort alors que seules les ampoules ayant les combinaisons : "VF, FV" & "VV, FF" seront reflétées dans le graphe d'implications. Nous remarquons ainsi le fait suivant : si nous démarrons d'un sommet en position ON, nous obtenons le même graphe que si nous avons démarré depuis le sommet en position OFF, à la différence que les noeuds du premier graphe sont dans un état opposés dans le second graphe et ce à condition de s'arrêter au préalable à chaque fois qu'une contradiction est rencontrée (nous considérons entre autre des arbres). En voici une illustration :

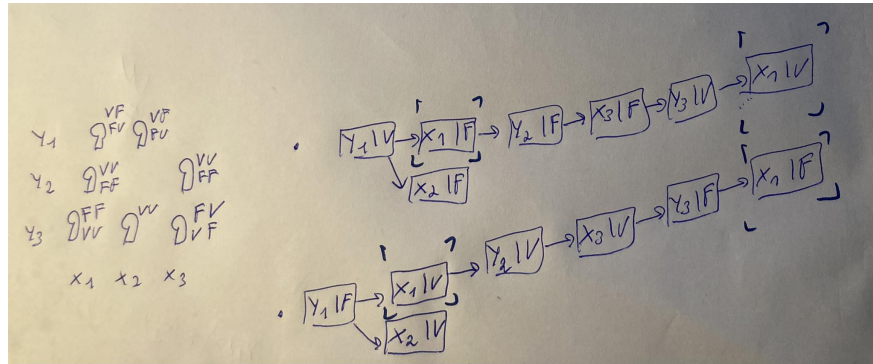


Figure 2: Graphes identiques à isomorphisme près

En partant de cette observation, nous en déduisons que si une contradiction est rencontrée, alors il est suffisant de conclure qu'on ne peut pas allumer toutes les ampoules, puisque si tel est le cas, partir du noeud opposé mènerait aussi à une contradiction. Ceci se produit lorsqu'il y a des ampoules ayant pour combinaisons : "VV, FF" et des ampoules ayant pour combinaisons "VF, FV". Donc cette approche est moins couteuse en terme d'arêtes et ne nécessite qu'un seul tour pour répondre négativement à la question. En revanche, s'il n'y pas de contradictions, alors il faut déterminer si pour les ampoules à combinaisons triviales, il est possible d'ajouter les arêtes correspondantes. S'il n'est pas possible de faire correspondre les combinaisons, il faut alors prendre le graphe opposé et procéder de manière similaire. Ainsi nous disposons d'un algorithme en $O(n + m)$ avec n le nombre d'implications non triviales et m le nombre de combinaisons triviales, Dans le meilleur des cas il est en $O(n)$ lorsque qu'il n'y a aucune ampoule à combinaison non triviale. Autrement la complexité en temps est similaire à l'algorithme précédent. Il est néanmoins bon de remarquer qu'il suffit de créer seulement une arête "dans un sens" pour notre deuxième algorithme.

3.2 Combien d'ampoules peut on allumer au maximum ?

3.2.1 Pas de réponse en temps polynomiale?

Nous reprenons quelques définitions données par Wikipedia English :

Définition 1 Walk : Il s'agit d'une suite finie ou infinie d'arêtes qui joignent des sommets.

Définition 2 Trail : il s'agit d'un walk tel que toutes les arêtes sont distinctes

définition 3 Path : il s'agit d'un trail où tous les sommets sont distincts.

Nous savons que pour les graphes dirigés acycliques (DAG) il existe un algorithme en temps linéaire pour trouver le plus long path. En revanche il s'agit d'un problème NP-Hard si le graphe est cyclique, ce qui, sans rentrer d'avantage dans les détails, est un problème ne pouvant pas se résoudre en temps polynomiale.

Le lien avec la question 2 est le suivant : maximiser le nombre d'ampoules revient à maximiser le nombre d'arêtes parcourues. Mais puisque nous avons à faire à des graphes pouvant contenir un cycle, nous pouvons nous ramener donc, à réduire notre problème en le problème suivant : Nous cherchons à trouver le plus long "walk" dans le graphe des implications, tel qu'il n'y a pas de contradictions. Nous avons aussi exploré la piste du problème dit : "max-cut-problem".

Il s'agit de trouver une partition des sommets du graphe en deux sous-ensembles complémentaires S et T tels que le nombre d'arêtes entre T et S est maximal. Bien que ce problème concerne initialement les graphes non dirigés, nous pouvons aussi le réduire à notre problème puisque les interrupteurs Y et les interrupteurs X forment en effet deux partitions disjointes telles que l'on cherche à maximiser le nombre d'arêtes entre elles.

3.2.2 Algorithme

Nous avons opté, compte tenu du fait qu'il s'agit d'un problème NP-HARD (pas d'algorithme en temps polynomial), de programmer un backtracking. Nous faisons essentiellement un parcours en largeur, et lorsqu'une contradiction a lieu, nous ne considérons pas l'arête donnant lieu à la contradiction, et nous continuons notre parcours. Notre fonction prendra en paramètre une liste de nœuds à ne pas parcourir, qui sont indépendants des éventuelles contradictions pouvant avoir lieu lors des parcours.

Le backtracking agit sur cette liste, et donc les éléments à ne pas parcourir changent aussi. Nous donnons une liste en la taille du nombre d'interrupteurs : en effet, si l'un des interrupteurs est en position éteinte, et qu'il est passé en paramètre de notre fonction, la fonction ne va pas parcourir ce nœud et donc, s'il n'y a pas une contradiction entre le nœud opposé à celui passé en paramètre, la fonction va parcourir son nœud opposé et donc nous livrera une autre solution. Nous sommes donc en $O(2^I \times (I + E))$ où I est le nombre d'interrupteurs et E le nombre d'arêtes, puisque nous effectuons un BFS pour chaque combinaison de boolean en la taille des interrupteurs.

3.2.3 Heuristique

Nous remarquons que certaines ampoules sont assurées d'être allumées, il s'agit des ampoules ayant pour combinaisons : "VF, FV" & "VV, FF". Nous savons déjà que certaines combinaisons comme : "FV", "VF, VV" ou "FF, FV" sont triviales dans le sens où il n'y a soit qu'une seule position des interrupteurs pouvant allumer l'ampoule, ou il suffit seulement que l'un des deux interrupteurs soit dans une certaine position. e.g. "VF, VV" Pour ce qui est des ampoules avec 3 combinaisons, aucune d'entre elles n'est triviale, toutefois sans perte de généralité et pour des raisons de simplicité, nous supposons que le tableau ne contient que des ampoules avec au plus 2 combinaisons. Nous fonctionnons ensuite de la sorte : Lorsque nous lisons notre fichier et construisons notre graphe, nous ne créons une arête que lorsqu'il s'agit d'une combinaison non triviale. e.g. "VV, FF" ou "VF, FV".

Ceci dit, nous remarquons donc un scénario similaire à la discussion sur les ampoules alignées horizontalement ou verticalement : Nous obtenons autant de sous graphes qu'il y a de composantes connexes, où à nouveau nous identifions les ampoules à combinaisons non triviales comme des nœuds, et imaginons des arcs les reliant si elles se trouvent en positions horizontales ou verticales l'une par rapport à l'autre. Remarquons, que pour chaque composante connexe, nous avons deux graphes d'implications. En effet, nous avons le choix de soit partir d'un nœud en position ON ou OFF. Il est intéressant de noter que les deux graphes obtenus sont "l'inverse l'un de l'autre" dans le sens où les deux graphes ont exactement la même structure et forme, à

la différence que les noeuds en position ON de l'un, sont les noeuds en position OFF de l'autre. Ainsi, si nous avons C composantes connexes, nous avons donc 2^C graphes possibles. Toutefois, des contradictions peuvent avoir lieu au sein des sous graphes, ce qui n'est pas réconfortant quant à la mise en oeuvre de l'algorithme. Pour palier à cela, nous n'ajoutons pas les noeuds qui poseraient une contradiction. Nous allons plus loin encore en ne créant pas une arête retour non plus, de sorte à avoir "un sous-arbre d'implications".

Une fois ceci fait, nous allons pour chacun des sous-arbres obtenus parcourir les combinaisons triviales et créer des arêtes s'il n'y a pas de contradictions. S'il y a T ampoules s'allumant sous des combinaisons triviales, nous avons donc un algorithme en $2^C \times T$ opérations. Evidemment, si le nombre d'ampoules à combinaisons triviales est faible, et qu'il n'y a que très peu de composantes connexes en termes d'ampoules à combinaisons non triviales, notre algorithme peut être en temps polynomial : en effet, notons :

C le nombre de composantes connexes d'ampoules à combinaisons non triviales,

I le nombre d'interrupteurs,

T le nombre d'ampoules à combinaisons non triviales et

$n := I + E$

Nous avons :

$$2^I \times n = 2^C \times T \Leftrightarrow 2^{(I-C)} \times n = T$$

Donc :

$$2^I \times n < 2^C \times T \iff T < 2^{(I-C)} \times n$$

Ainsi, pour C petit, et

$$T < 2^{(I-C)} \times n$$

nous disposons donc d'un algorithme nettement plus performant.